

认知科学与类脑计算

主讲教师：刘千惠

联系方式：qhliu@sdu.edu.cn

山东大学 人工智能学院

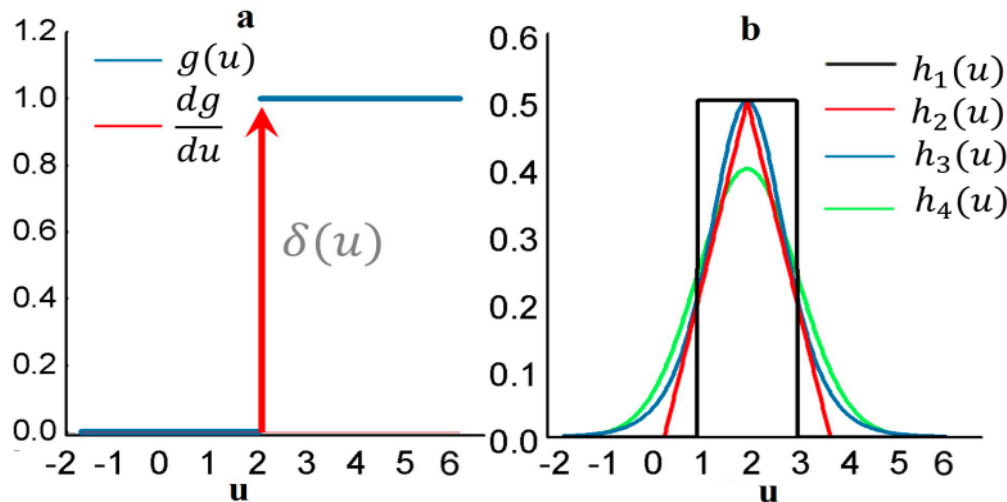


Spatio-Temporal Backpropagation (STBP) 算法

3. 代理梯度函数

脉冲不可微问题：神经元激活函数 $g(u)$ 是 $\delta(u)$ 的不可微狄拉克函数，该函数的值除了在零点处无穷大以外，其他地方的值都为零，这就导致了梯度消失或爆炸问题，从而使得误差误差传播。

➤ 为了解决该问题，STBP算法引入了四条曲线来用于近似表示脉冲激活函数的导数。



Spatio-Temporal Backpropagation (STBP) 算法

$$h_1(u) = \frac{1}{a_1} \text{sign}(|u - V_{th}| < \frac{a_1}{2})$$

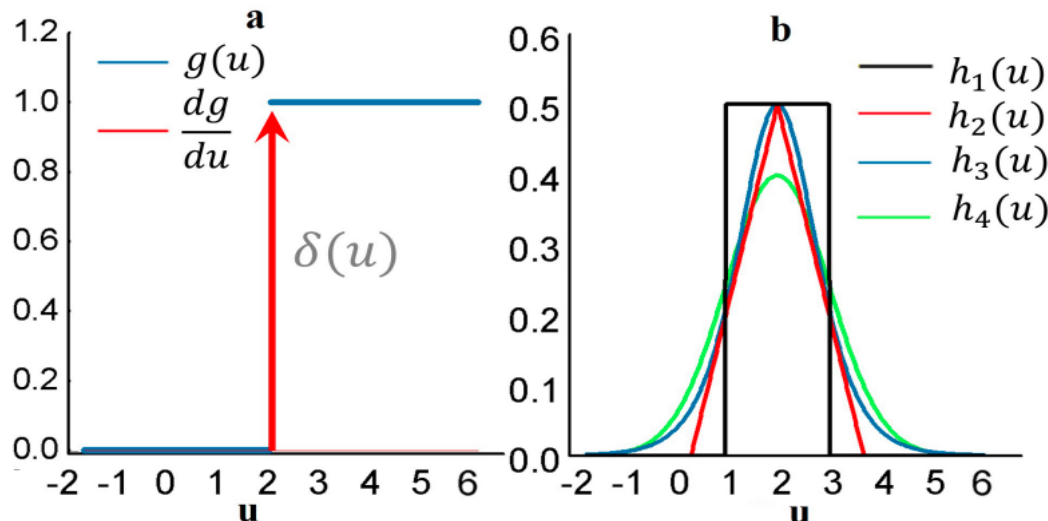
$$h_2(u) = (\frac{\sqrt{a_2}}{2} - \frac{a_2}{4}|u - V_{th}|) \text{sign}(\frac{2}{\sqrt{a_2}} - |u - V_{th}|)$$

$$h_3(u) = \frac{1}{a_3} \frac{e^{\frac{V_{th}-u}{a_3}}}{(1+e^{\frac{V_{th}-u}{a_3}})^2}$$

$$h_4(u) = \frac{1}{\sqrt{2\pi a_4}} e^{-\frac{(u-V_{th})^2}{a_4}}$$

四种曲线分别是矩形函数、多项式函数、sigmoid函数和高斯积累分布函数的导数。参数 α_i 决定曲线的尖锐程度。

Spatio-Temporal Backpropagation (STBP) 算法



这些函数都有共同特点：

1. 中心在阈值 V_{th} 附近
2. 远离阈值时接近0
3. 阈值附近有较大值
4. 积分面积为 1

因此它们可以看作 Dirac 函数的不同近似。

Spatio-Temporal Backpropagation (STBP) 算法实现

1. 迭代LIF模型

```
# membrane potential update
def mem_update(ops, x, mem, spike):
    mem = mem * decay * (1. - spike) + ops(x)
    spike = act_fun(mem) # act_fun : approximation firing function
    return mem, spike
```

$$\begin{aligned} u^{t+1,n} &= u^{t,n} f(o^{t,n}) + x^{t+1,n} \\ u^{t+1,n} &= \tau u^{t,n} (1 - o^{t,n}) + x^{t+1,n} \\ o^{t+1,n} &= g(u^{t+1,n}) \end{aligned} \quad \begin{aligned} f(o_i^{t,n}) &\approx \begin{cases} \tau, & o_i^{t,n} = 0 \\ 0, & o_i^{t,n} = 1 \end{cases} \\ g(u) &= \begin{cases} 1, & u \geq V_{th} \\ 0, & u < V_{th} \end{cases} \end{aligned}$$

act_fun 在前向传播中执行阈值发放；在反向传播中使用 surrogate gradient 近似其导数

Spatio-Temporal Backpropagation (STBP) 算法实现

2. 前向与反向传播训练

```
class ActFun(torch.autograd.Function):  
  
    @staticmethod  
    def forward(ctx, input):  
        ctx.save_for_backward(input)  
        return input.gt(thresh).float()  
  
    @staticmethod  
    def backward(ctx, grad_output):  
        input, = ctx.saved_tensors  
        grad_input = grad_output.clone()  
        temp = abs(input - thresh) < lens  
        return grad_input * temp.float()  
  
act_fun = ActFun.apply
```

Forward:

- 把膜电位保存起来，留给反向传播使用。
- 前向传播用硬阈值产生 0/1 spike

Backward:

- 取出前向保存的膜电位
- 复制上游梯度 $\frac{\partial L}{\partial o}$
- 判断膜电位是否在阈值附近:

如果满足, temp=1;

否则, temp=0。

也就是用矩形窗口代理梯度

$$\frac{\partial L}{\partial u} = \frac{\partial L}{\partial o} \cdot h(u) \quad \frac{\partial L}{\partial u} = \frac{\partial L}{\partial o} \cdot \mathbf{1}(|u - V_{th}| < lens)$$

$$\alpha_i = 2 * lens$$

Spatio-Temporal Backpropagation (STBP) 算法实现

3. 网络结构

```
class SCNN(nn.Module):
    def __init__(self):
        super(SCNN, self).__init__()
        in_planes, out_planes, stride, padding, kernel_size = cfg_cnn[0]
        self.conv1 = nn.Conv2d(in_planes, out_planes, kernel_size=kernel_size, stride=stride, padding=padding)
        in_planes, out_planes, stride, padding, kernel_size = cfg_cnn[1]
        self.conv2 = nn.Conv2d(in_planes, out_planes, kernel_size=kernel_size, stride=stride, padding=padding)

        self.fc1 = nn.Linear(cfg_kernel[-1] * cfg_kernel[-1] * cfg_cnn[-1][1], cfg_fc[0])
        self.fc2 = nn.Linear(cfg_fc[0], cfg_fc[1])
```

这段代码是在定义一个 SNN 卷积网络结构，类名是 SCNN。它只展示了 `__init__` 初始化部分，也就是在创建网络时声明有哪些层。整体结构是：

Conv1→Conv2→FC1→FC2

也就是两个卷积层加两个全连接层。

3. 网络结构

```
class SCNN(nn.Module):
    def __init__(self):
        self.fc2 = nn.Linear(cfg_fc[0], cfg_fc[1])

    def forward(self, input, time_window = 20):
        c1_mem = c1_spike = torch.zeros(batch_size, cfg_cnn[0][1], cfg_kernel[0], cfg_kernel[0], device=device)
        c2_mem = c2_spike = torch.zeros(batch_size, cfg_cnn[1][1], cfg_kernel[1], cfg_kernel[1], device=device)

        h1_mem = h1_spike = h1_sumspike = torch.zeros(batch_size, cfg_fc[0], device=device)
        h2_mem = h2_spike = h2_sumspike = torch.zeros(batch_size, cfg_fc[1], device=device)

        for step in range(time_window): # simulation time steps
            x = input > torch.rand(input.size(), device=device) # prob. firing

            c1_mem, c1_spike = mem_update(self.conv1, x.float(), c1_mem, c1_spike)

            x = F.avg_pool2d(c1_spike, 2)

            c2_mem, c2_spike = mem_update(self.conv2, x, c2_mem, c2_spike)

            x = F.avg_pool2d(c2_spike, 2)
            x = x.view(batch_size, -1)

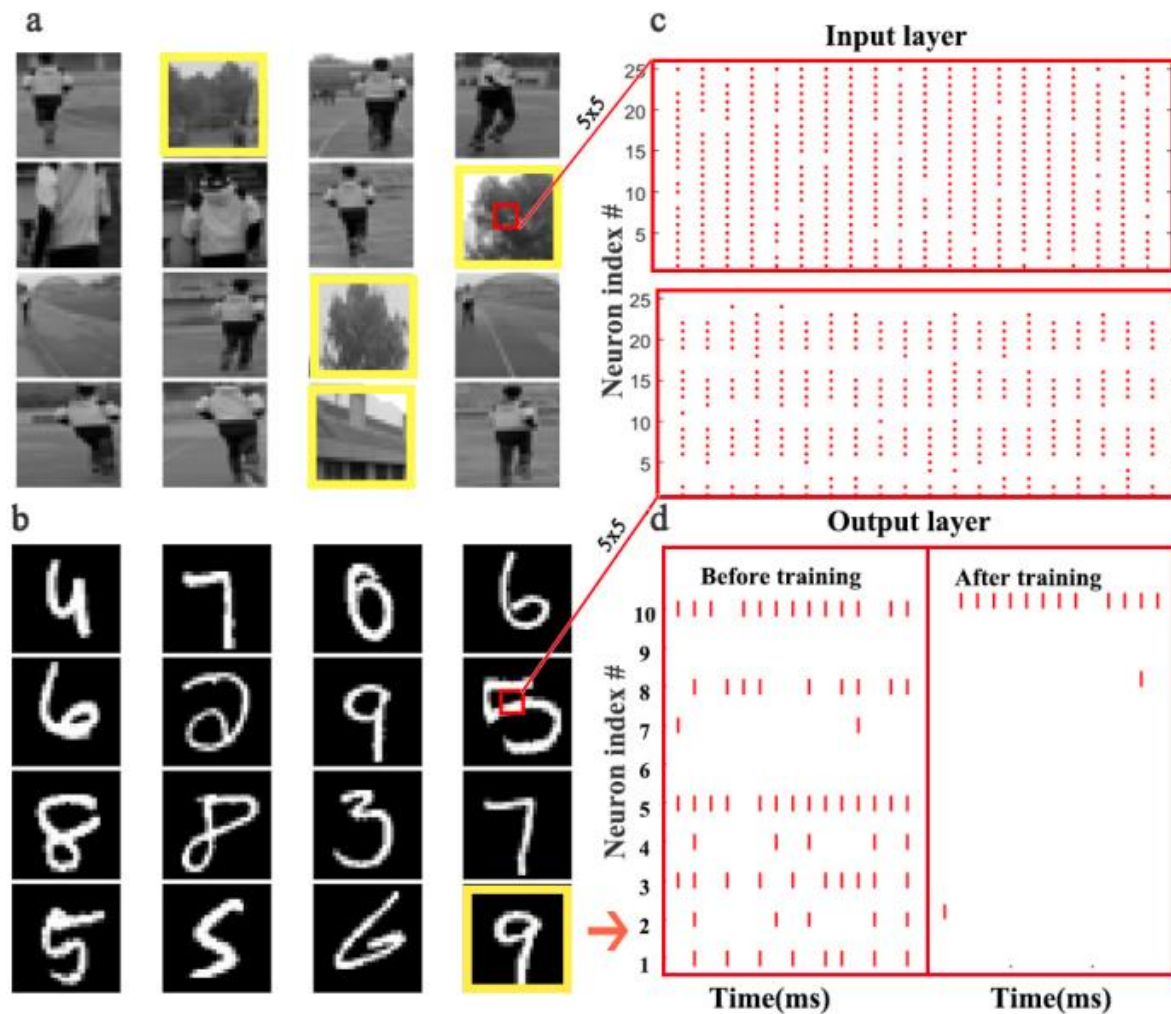
            h1_mem, h1_spike = mem_update(self.fc1, x, h1_mem, h1_spike)
            h1_sumspike += h1_spike
            h2_mem, h2_spike = mem_update(self.fc2, h1_spike, h2_mem, h2_spike)
            h2_sumspike += h2_spike

        outputs = h2_sumspike / time_window
        return outputs
```

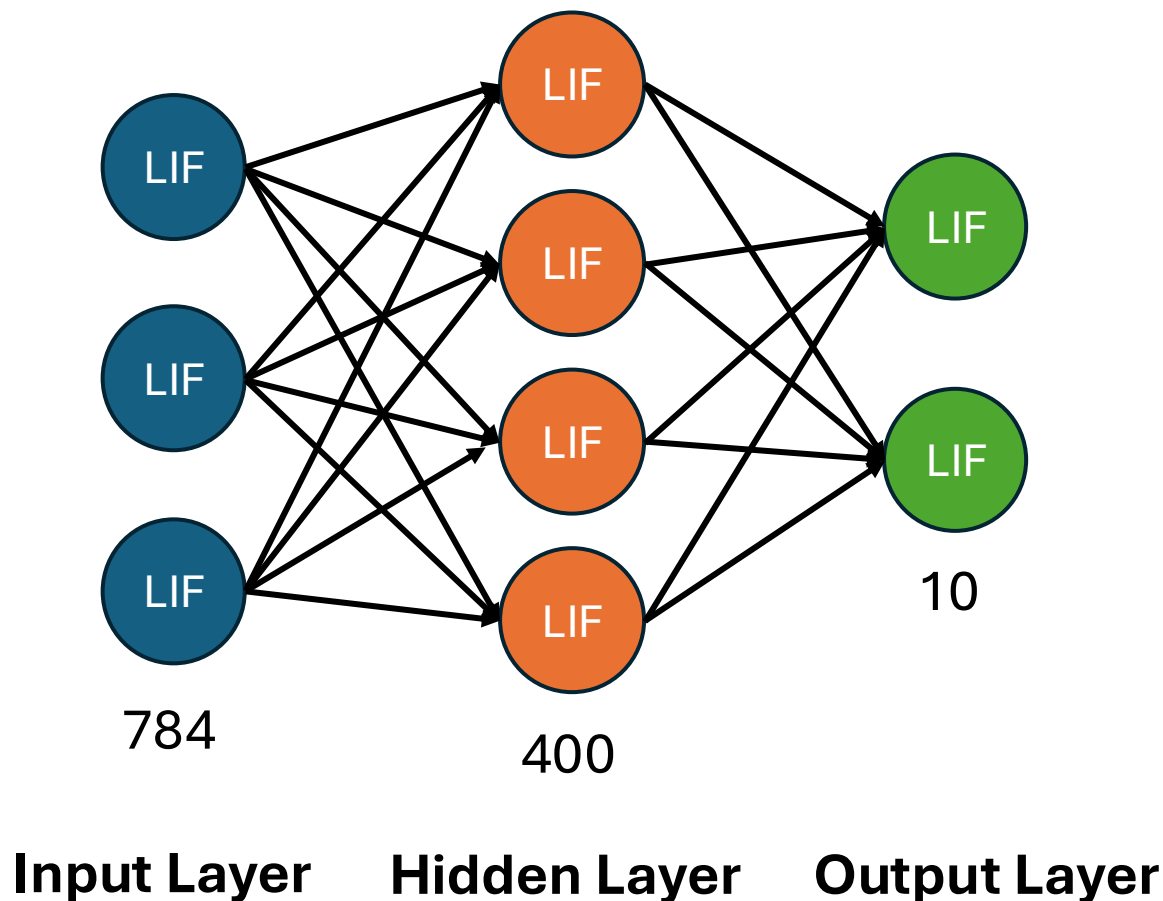
在多个时间步内反复模拟LIF神经元的膜电位更新和spike发放，最后用输出层spike的平均发放率作为分类结果。

- 初始化膜电位和spike
- 时间循环
- 输入编码：概率发放
- 前向过程
- 输出结果：平均发放率

Spatio-Temporal Backpropagation (STBP) 应用实例



Spatio-Temporal Backpropagation (STBP) 应用实例



Spatio-Temporal Backpropagation (STBP) 应用实例

TABLE II: Comparison with the state-of-the-art spiking networks with similar architecture on MNIST.

Model	Network structure	Training skills	Accuracy
Spiking RBM (STDP) [31]	784-500-40	None	93.16%
Spiking RBM(pre-training*) [20]	784-500-500-10	None	97.48%
Spiking MLP(pre-training*) [19]	784-1200-1200-10	Weight normalization	98.64%
Spiking MLP(BP) [22]	784-200-200-10	None	97.66%
Spiking MLP(STDP) [15]	784-6400	None	95.00%
Spiking MLP(BP) [23]	784-800-10	Error normalization/ parameter regularization	98.71%
Spiking MLP(STBP)	784-800-10	None	98.89%

TABLE III: Comparison with the typical MLP over object detection dataset.

Model	Network structure	Accuracy	
		Mean	Interval*
Non-spiking MLP(BP)	784-400-10	98.31%	[97.62%, 98.57%]
Spiking MLP(STBP)	784-400-10	98.34%	[97.94%, 98.57%]

Spatio-Temporal Backpropagation (STBP) 应用实例

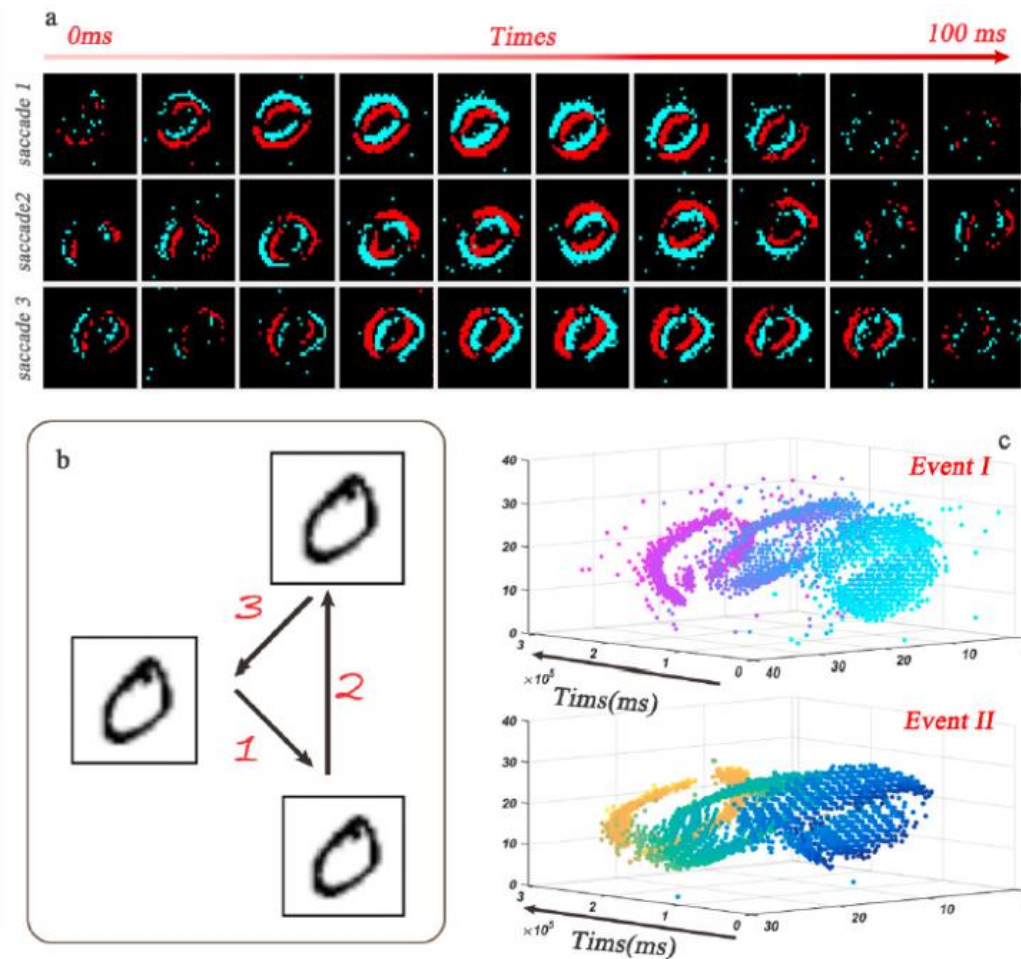
TABLE V: Comparison with other spiking CNN over MNIST.

Model	Network structure	Accuracy
Spiking CNN (pre-training*) [13]	$28 \times 28 \times 1$ -12C5-P2-64C5-P2-10	99.12%
Spiking CNN(BP) [23]	$28 \times 28 \times 1$ -20C5-P2-50C5-P2-200-10	99.31%
Spiking CNN (STBP)	$28 \times 28 \times 1$ -15C5-P2-40C5-P2-300-10	99.42%

TABLE VI: Comparison with the typical CNN over object detection dataset.

Model	Network structure	Accuracy	
		Mean	Interval*
Non-spiking CNN(BP)	$28 \times 28 \times 1$ -6C3-300-10	98.57%	[98.57%, 98.57%]
Spiking CNN(STBP)	$28 \times 28 \times 1$ -6C3-300-10	98.59%	[98.26%, 98.89%]

Spatio-Temporal Backpropagation (STBP) 应用实例

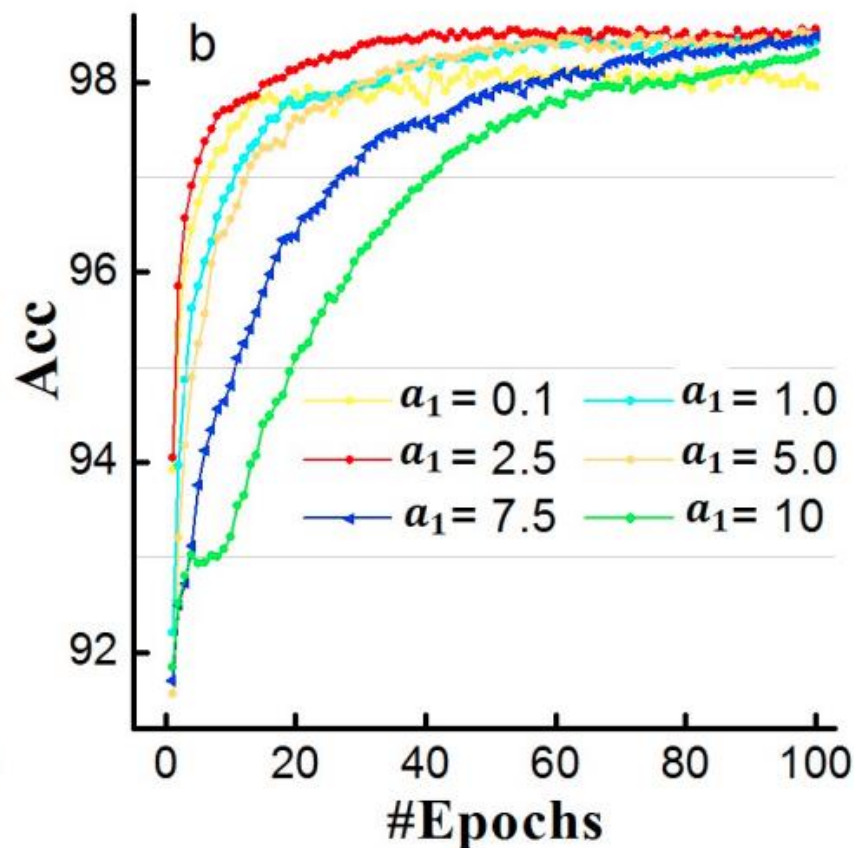
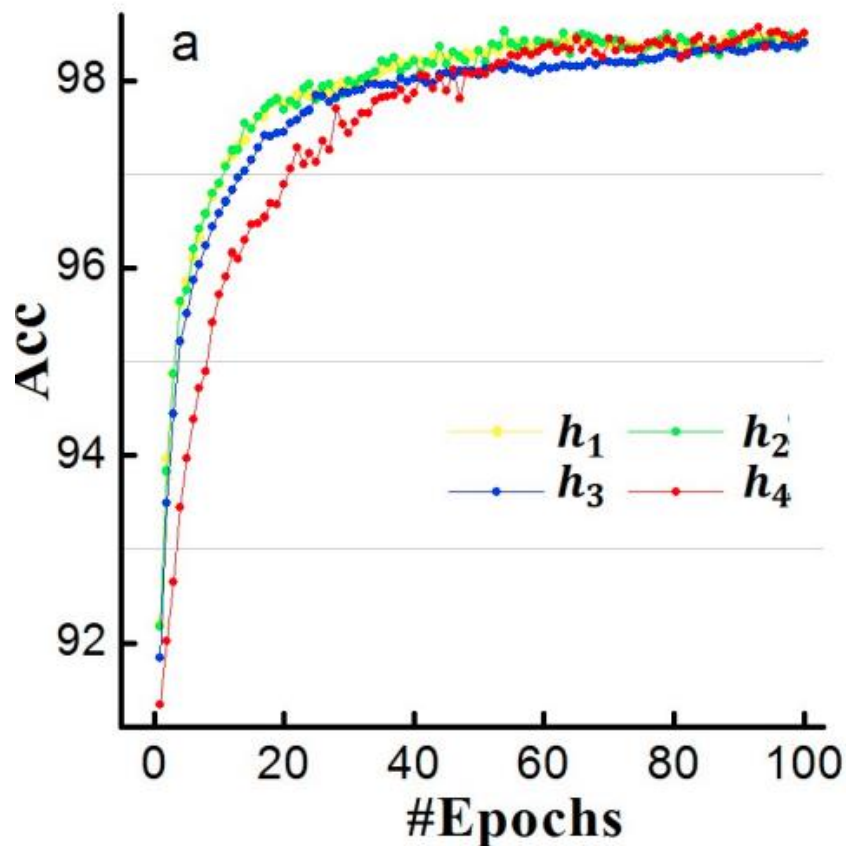


Spatio-Temporal Backpropagation (STBP) 应用实例

TABLE IV: Comparison with state-of-the-art networks over N-MNIST.

Model	Network structure	Training skills	Accuracy
Non-spiking CNN(BP) [24]	-	None	95.30%
Non-spiking CNN(BP) [34]	-	None	98.30%
Non-spiking MLP(BP) [23]	$34 \times 34 \times 2-800-10$	None	97.80%
LSTM(BPTT) [24]	-	Batch normalization	97.05%
Phased-LSTM(BPTT) [24]	-	None	97.38%
Spiking CNN(pre-training*) [34]	-	None	95.72%
Spiking MLP(BP) [23]	$34 \times 34 \times 2-800-10$	Error normalization/ parameter regularization	98.74%
Spiking MLP(BP) [35]	$34 \times 34 \times 2-10000-10$	None	92.87%
Spiking MLP(STBP)	$34 \times 34 \times 2-800-10$	None	98.78%

Spatio-Temporal Backpropagation (STBP) 应用实例



其他基于代理梯度的学习算法:

1. SLayer针对脉冲响应模型, 设计了一种同时学习突触权重和轴突延迟的SNN反向传播机制, 并基于脉冲状态变化的概率分布特性, 提出采用近似指数衰减代理梯度函数, 来解决脉冲发放不可微的问题。

Shrestha S B, Orchard G. Slayer: Spike layer error reassignment in time[J]. Advances in neural information processing systems, 2018, 31.

2. STCA算法针对具有延迟反馈的时间信用分配问题, 提出了基于脉冲簇的全新损失函数, 并推导了具有复杂时空动态C-LIF神经元的时空信用分配计算算法。

Gu P, Xiao R, Pan G, et al. STCA: Spatio-temporal credit assignment with delayed feedback in deep spiking neural networks[C]//IJCAI. 2019, 15: 1366-1372.

3. 自适应平滑梯度学习 (ASGL) 针对代理梯度导致的前馈-反馈的不匹配问题, 提出脉冲实值混合掩蔽训练策略。该策略可以在训练过程中, 逐步将实值神经网络的误差转换为脉冲神经网络的误差, 进而计算出共享权重的内嵌脉冲神经网络的精确梯度。

Wang Z, Jiang R, Lian S, Yan R, Tang H. Adaptive smoothing gradient learning for spiking neural networks[C]//ICML. 2023 3: 35798-35816.

基于脉冲发放时间的反传算法

基于脉冲发放时间的反传算法，其梯度传播具有显著特性：仅通过神经元发放的脉冲传播损失函数带来的梯度信号，这类算法将脉冲发放时间对神经元膜电位的导数近似为神经元膜电位对时间求导的负倒数。

这种学习机制可以充分发挥脉冲的异步计算优势，一方面降低了训练计算资源消耗，另一方面还提高了信息处理的速度和效率。

基于脉冲发放时间的反传算法

时间脉冲序列学习算法（TSSL-BP）提出了一种新的时间脉冲序列学习来训练深度SNN的方法。其核心创新点在于，通过考虑神经元发放脉冲的两种时间依赖关系，即神经元间和神经元内的依赖关系，推导出脉冲活动在空间和时间更加精确的梯度反传，实现高精度地学习给定的目标序列。

Wenrui Zhang and Peng Li. Temporal spike sequence learning via backpropagation for deep spiking neural networks. *Advances in Neural Information Processing Systems*, pages 12022–12033, 2020.

事件驱动反向传播算法（EDBP）证明了在SNN中任意一层所有神经元发放的所有脉冲上所携带的梯度总和，在层与层之间保持不变，不会因层间传递而增减。针对平均池化和脉冲响应核函数反向梯度会破坏上述梯度总和不变性破坏问题，EDBP算法调整了平均池化的反传策略并设计了新的反向传播核，为SNN时间反向传播训练的稳定学习提供了理论保障。

Zhu, Yaoyu, et al. "Training spiking neural networks with event-driven backpropagation." *Advances in Neural Information Processing Systems* 35, pages 30528-30541, 2022

脉冲神经网络监督学习算法

- 经典人工神经网络的学习算法回顾
- 单层脉冲神经网络的学习算法
- 深度脉冲神经网络的学习算法
- 转换算法（ANN-SNN Conversion）

转换算法（ANN-SNN Conversion）

在深度脉冲神经网络（Deep SNNs, DSNNs）的训练中，直接进行反向传播，常常会面临梯度消失、梯度爆炸、时间维度依赖和计算开销大等问题。为应对上述直接训练挑战，研究人员提出了 ANN-SNN 转化算法，该算法一种有效的间接训练方法。ANN-SNN 转化算法的核心思想是：首先使用反向传播算法训练一个神经网络（ANN），然后将训练好的权重映射到结构等价的脉冲神经网络（SNN）中，而后利用SNN进行推理任务。

ANN-SNN 转化基本原理

ANN中激活函数的激活值可以近似等价于SNN中神经元的脉冲发放速率（Firing Rate）。在ANN中，神经元的激活值是一个连续变量，通常通过激活函数（如ReLU、Sigmoid等）进行非线性映射。而在SNN中，神经元以稀疏离散的脉冲的形式发放信号，其中脉冲发放速率可以看作是ANN中连续激活值的离散时间编码版本。因此，如果在训练ANN时对输入、权重、激活值等进行适当的约束，便可以实现ANN到SNN的映射，转化完成后，可利用SNN 的事件驱动特性（仅脉冲发放时耗能），在推理阶段实现低功耗、高效率的信号处理。

ANN-SNN 转化基本原理

人工神经元通过乘积累加运算以及非线性激活函数输出激活值，网络前馈过程如下：

$$\mathbf{a}^l = f(\mathbf{W}^{l-1} \mathbf{a}^{l-1} + \mathbf{b}^l)$$

脉冲神经元输出的是 spike，所以不能直接像 ANN 那样输出连续激活值，于是用一段时间窗口内的平均发放率表示神经元输出：

$$r_{si}^l = \frac{n_{si}^l}{T}$$

ANN-SNN 转化基本原理

为了让 神经元发放率更接近 ReLU结果，假设它没有 refractory period和leaky，这样输入越强，单位时间内可以产生更多 spike。

在T个时间步内，脉冲神经元i所发放的脉冲的数量可以从将所有累积的膜电压除以其阈值得到：

$$\begin{aligned} n_{s_i}^{l+1} &= \left\lfloor \frac{\sum_{t=1}^T \sum_{j=1}^n w_{ij} s_j^l(t)}{\theta_i} \right\rfloor \\ &= \frac{\sum_{t=1}^T \sum_{j=1}^n w_{ij} s_j^l(t)}{\theta_i} - \Delta\theta \end{aligned}$$

其中 $\Delta\theta$ 是近似误差项，由残余膜电位等导致。如果 T 很大，这个误差对平均发放率的影响会变小。

ANN-SNN 转化基本原理

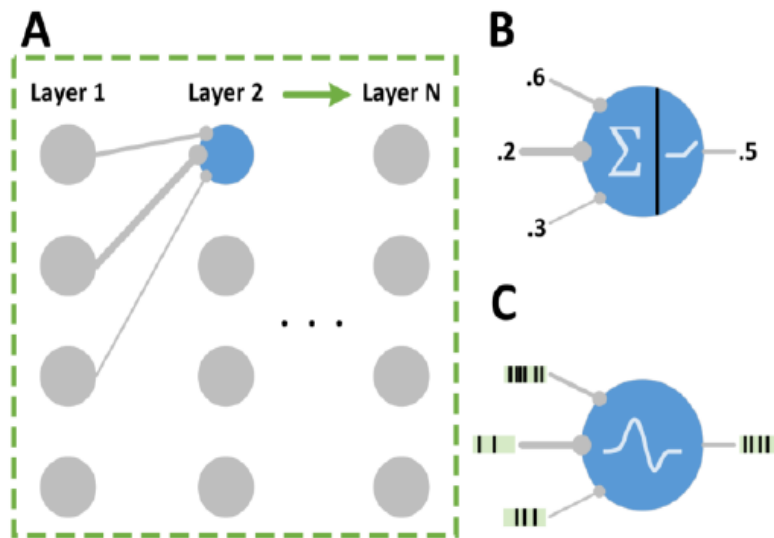
脉冲发放频率 $r_{s_i}^{l+1}$ 为:

$$\begin{aligned} r_{s_i}^{l+1} &= \frac{n_{s_i}^{l+1}}{T} = \frac{\sum_{t=1}^T \sum_{j=1}^n w_{ij} s_j^l(t)}{\theta_i \times T} - \frac{\Delta\theta}{T} \\ &= \frac{\sum_{j=1}^n w_{ij} \sum_{t=1}^T s_j^l(t)}{\theta_i \times T} - \frac{\Delta\theta}{T} \\ &= \frac{\sum_{j=1}^n w_{ij} \left[\frac{\sum_{t=1}^T s_j^l(t)}{T} \right]}{\theta_i} - \frac{\Delta\theta}{T} \\ &= \frac{\sum_{j=1}^n w_{ij} r_{s_j}^l}{\theta_i} - \frac{\Delta\theta}{T} \\ &\approx \frac{\sum_{j=1}^n w_{ij} r_{s_j}^l}{\theta_i} = \frac{W_i R_s^l}{\theta_i} \end{aligned}$$

ANN-SNN 转化基本原理

训练好的ANN 和转换后的SNN 保持结构与权重一致性。**ANN**神经元通过计算输入的加权和，之后进行ReLU 运算产生输出，脉冲神经元在突触前脉冲到达时更新状态，并在膜电压超过阈值时产生脉冲来传递信息。

两类神经元的动态特性不同，但如果它们的输入值相等，输出值就会因为相同的连接权重和网络结构而几乎相同，即所有ANN神经元的激活值均能通过对应SNN神经元的平均发放速率近似表征，且网络结构与权重的一致性确保了层间信号传递的近似性。



ReLU 的 ANN 和脉冲神经网络之间的关系

- ReLU 可以被视为无不应期的 IF 神经元的放电率近似值，其中 ReLU 的输出与 IF 神经元在给定时间窗口内产生的脉冲数成正比。ReLU 使得权重更新的形式特别简单。
- 对于分类任务，只有输出层中所有单元的最大激活值才重要，这使得整体速率可以按一个常数因子进行缩放。
- 由于没有偏置来提供外部参考值，神经元权重彼此之间以及与神经元阈值之间的相对比例是唯一重要的参数。

ANN-SNN 转换流程

1. 对网络的所有单元使用 ReLU。
2. 在整个训练过程中将偏置固定为零，并使用反向传播进行训练。
3. 将 ReLU 网络的权重直接映射到 IF 单元网络。
4. 使用权重归一化以获得接近无损的精度和更快的收敛速度。

转换后精度损失的三个来源

- 欠激活：神经元收到的输入太弱，膜电位达不到阈值，因此 spike 发放率偏低。
- 过激活：神经元收到的输入太强，导致根据 ReLU 值在单个时间步内发放超过一个 spike，但 SNN 每个时间步最多只能发一个 spike。
- 脉冲随机性：spike 输入通常是概率生成的，即使平均发放率正确，短时间内也可能出现某些时间 spike 太密集或者某些时间 spike 太稀疏。

所有这些误差都可以通过找到合适的 spike 阈值、输入权重和输入发放率之间的平衡来缓解。

阈值过高或权重过低:

- 可以减少过度发放;
- 可以减轻 spike 非理想性带来的过激活;
- 容易造成激活不足。

阈值过低或权重过高:

- 减少激活不足;
- 神经元更容易产生 spike;
- 容易过度发放。

解决方法:

- 手动调节每一层的阈值和权重比例
- 计算每层权重的 rescaling factor, 使得 ANN 激活和 SNN 发放率尽量匹配。

基于模型的归一化 Model-Based Normalization

只看网络权重，不需要数据

Algorithm 1: Model-Based Normalization

```
1 | for layer in layers:
2 |     max_pos_input = 0
3 |     # Find maximum input for this layer
4 |     for neuron in layer.neurons:
5 |         input_sum = 0
6 |         for input_wt in neuron.input_wts:
7 |             input_sum += max(0, input_wt)
8 |         max_pos_input = max(max_pos_input, input_sum)
9 |     # Rescale all weights
10 |    for neuron in layer.neurons:
11 |        for input_wt in neuron.input_wts:
12 |            input_wt = input_wt / max_pos_input
```

归一化后，任何神经元即使在所有正输入 spike 同时到达的最坏情况下，单步输入也不会超过阈值，因此不会需要在一个时间步内表示超过一个 spike。但可能需要更长的时间生成脉冲。

基于数据的归一化 Data-Based Normalization

在真实数据输入下，每一层通常会产生多大的激活。

Algorithm 2: Data-Based Normalization

```
1 previous_factor = 1
2 for layer in layers:
3     max_wt = 0
4     max_act = 0
5     for neuron in layer.neurons:
6         for input_wt in neuron.input_wts:
7             max_wt = max(max_wt, input_wt)
8             max_act = max(max_act, neuron.output_act)
9     scale_factor = max(max_wt, max_act)
10    applied_factor = scale_factor / previous_factor
11    # Rescale all weights
12    for neuron in layer.neurons:
13        for input_wt in neuron.input_wts:
14            input_wt = input_wt / applied_factor
15    previous_factor = scale_factor
```

max_act 控制整体发放率不超过可表示范围， max_wt 防止单个输入 spike 通过过大的权重造成瞬时过强输入；取二者最大值，是同时满足这两个约束的最小安全缩放因子。

$$W^l \leftarrow \frac{W^l}{\text{applied_factor}}$$

$$\text{applied_factor} = \frac{\text{scale_factor}}{\text{previous_factor}} = \frac{\lambda^l}{\lambda^{l-1}}$$

$$W^l \leftarrow W^l \frac{\lambda^{l-1}}{\lambda^l}$$

基于数据的归一化 Data-Based Normalization

假设 ANN 中有两层:

$$a^1 = W^1 x$$

$$\lambda^1 = 5, \quad \lambda^2 = 10$$

$$a^2 = W^2 a^1$$

$$r^1 \approx \frac{a^1}{5}$$

目标是让 SNN 的每层发放率表示归一化后的 ANN 激活:

$$r^2 \approx \frac{a^2}{10}$$

第一层权重: SNN 里第二层实际接收的是:

$$\tilde{W}^1 = \frac{W^1}{5} \quad r^1 \approx \frac{a^1}{5}$$

如果第二层权重也直接除以自己的scale=10:

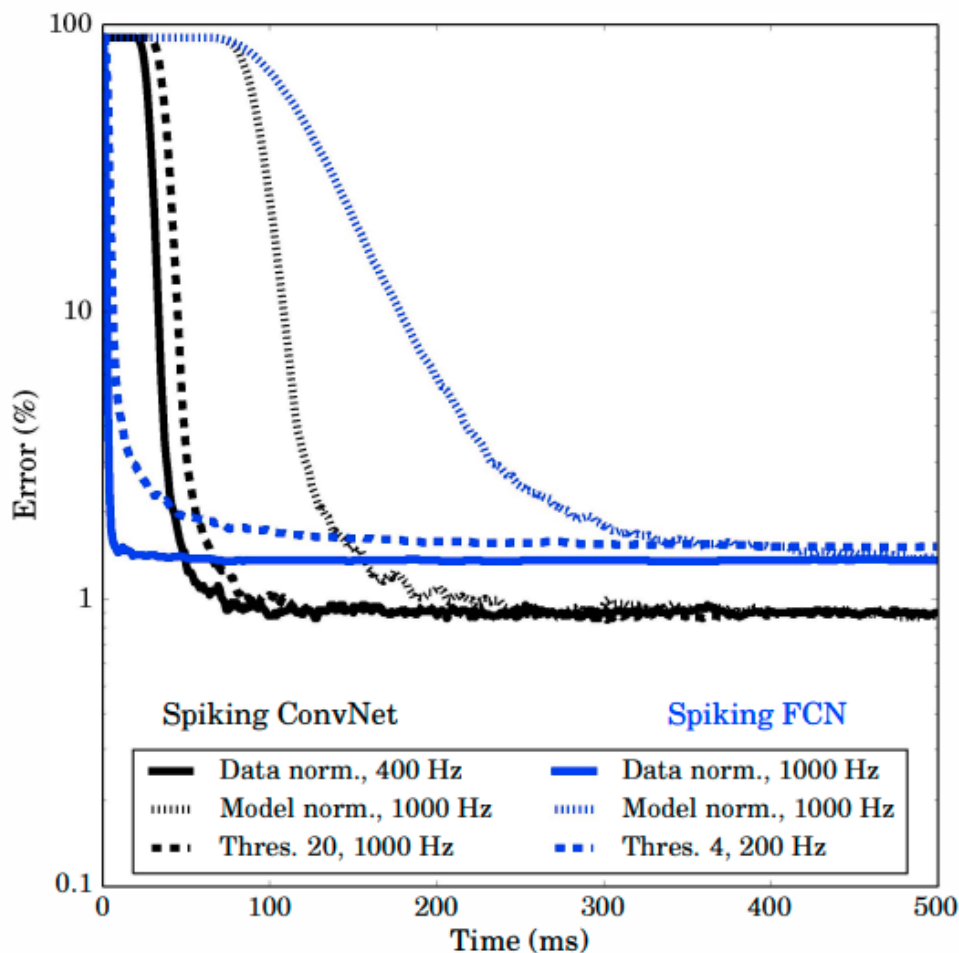
$$\tilde{W}^2 = \frac{W^2}{10} \quad \tilde{W}^2 r^1 = \frac{W^2}{10} \cdot \frac{a^1}{5} = \frac{W^2 a^1}{50} = \frac{a^2}{50}$$

那么第一层输出发放率近似:

因此需要previous_factor:

$$r^1 \approx \frac{a^1}{5} \quad \tilde{W}^2 = W^2 \frac{\lambda^1}{\lambda^2} = W^2 \frac{5}{10} = \frac{W^2}{2} \quad \tilde{W}^2 r^1 = \frac{W^2}{2} \cdot \frac{a^1}{5} = \frac{W^2 a^1}{10} = \frac{a^2}{10}$$

ANN-SNN 转换结果



分类误差随时间的变化。

黑色曲线显示卷积神经网络的结果，蓝色曲线显示全连接网络的结果。

除了模型归一化网络外，所有网络在 100 毫秒内都表现出非常低的误差。全连接数据归一化网络在 6 毫秒后就接近其峰值准确率（误差为 1.74%）。

ANN-SNN 转换

方法	依据	优点	缺点
Model-Based	只看权重最大正输入	安全，无需数据	过于保守，可能发放不足
Data-Based	用训练集统计数据最大激活	精度高，延迟低	依赖训练集代表性

ANN-SNN 转换结果

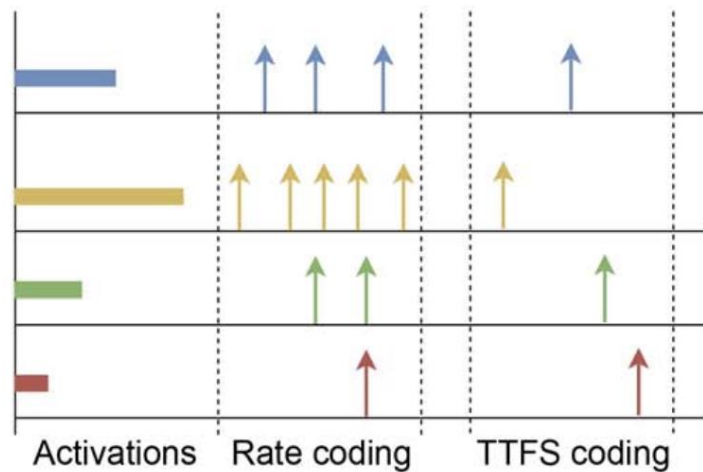
TABLE I: Comparison of classification accuracy for different network architectures and conversion mechanisms.

Network Type	Input Rate	Thr.	Accuracy
ConvNets			
ReLU Rate-Based	—	—	99.14%
IF Network	1000 Hz	20.0	99.12%
Data-Norm. Net	400 Hz	1.0	99.10%
Model-Norm. Net	1000 Hz	1.0	99.11%
FC Networks			
ReLU Rate-Based	—	—	98.68%
IF Network	200 Hz	4.0	98.48%
Data-Norm. Net	1000 Hz	1.0	98.64%
Model-Norm. Net	1000 Hz	1.0	98.61%

TTFS编码的ANN-SNN 转换

与传统 rate-based ANN-SNN 转换不同，TTFS 转换不是用一段时间窗口内的 spike 数量或平均发放率来近似 ANN 的激活值，而是利用脉冲到达时间来近似ANN 激活值。

在 TTFS coding 中，每个神经元在一个时间窗口内通常只发放一次 spike，激活值越大，对应的首次发放时间越早。这样，TTFS 可以用更少的 spike 表示信息，从而降低计算量和能耗。



TTFs编码的ANN-SNN 转换

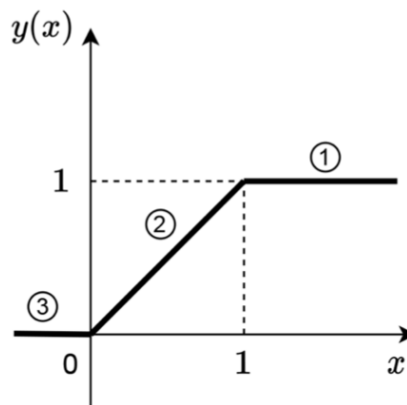
ANN 神经元模型

在 ANN 中，神经元仍然是普通的模拟值神经元，其输出由加权求和再经过激活函数得到：

$$a_i^l = f \left(\sum_j w_{ij}^l a_j^{l-1} + b_i^l \right)$$

其中，f函数不是使用普通 ReLU，而是使用 ReLU1，也就是把激活值限制在 [0,1]内：

$$f(x) = \begin{cases} 0, & x \leq 0 \\ x, & 0 < x \leq 1 \\ 1, & x > 1 \end{cases}$$



TTFS编码的ANN-SNN 转换

TTFS 用“首次发放时间”表示 ANN 激活值，而一个固定时间窗口只能表示有限范围内的值。所以需要把 ANN 激活值限制到 $[0,1]$ ，才能和 spike time 建立稳定的一一对应关系。

ANN 激活值 α_i^l	SNN spike time t_i^l	含义
1	窗口起点	最大激活，最早发放
0.5	窗口中间	中等激活
0	窗口终点	最小激活，最晚发放